

Uživatelský manuál Marvin

Kompletní průvodce lokální chatovací a vývojovou stanicí

Verze aplikace	1.8.2
Uživatelský manuál	2026

Obsah

1. Co je Marvin	3
2. Požadavky a instalace	3
3. Prohlídka desktopového rozhraní	5
4. Modely, KV cache, kontext a thinking	7
5. Odesílání, steering, Stop a průběh	8
6. Pracovní režimy	9
7. Projekty a workspace	9
8. Správa chatů	9
9. Kontext, komprese a pins	10
10. Tři vrstvy paměti	10
11. Volitelné skills	10
12. Internet a výzkum	11
13. Psaní a export dokumentů	11
14. Vývojový workflow	12
15. Ovládání počítače	13
16. Autonomie a potvrzování	14
17. Terminálové UI a CLI	14
18. Úložiště, zálohy a logy	15
19. Řešení problémů	16
20. Reference nástrojů	17
21. Praktické příklady	18
22. Provozní omezení	19
23. Rozšířené pracovní operace	19

1. Co je Marvin

Marvin je soukromá desktopová aplikace pro Windows, která provozuje lokální jazykové modely jako obecný chat, výzkumného asistenta, partnera pro psaní, coding agenta a operátora počítače. Inference modelu běží na pracovní stanici přes llama.cpp; konverzace a projektové soubory zůstávají na lokálním disku.

Aplikace je navržena pro jednoho uživatele a jeden aktivní model. Nepoužívá paralelní modelové agenty. Aktivní model ale může paralelně spouštět nezávislé čtecí nástroje a může ponechat dlouhé příkazy běžet na pozadí.

POZNÁMKA: Inference modelu je lokální. Když model použije web_search nebo web_fetch, aplikace provádí běžné internetové požadavky na vyhledávače a veřejné webové stránky. Pokud potřebujete zcela offline relaci, vypněte web v config.yaml nebo nepoužívejte výzkumné/webové požadavky.

Hlavní schopnosti

- Běžná konverzace bez programátorského chování.
- Výzkum z více webových a dokumentových zdrojů se závěrečnou syntézou.
- Psaní a revize scénářů, článků, reportů, treatmentů a dalších dokumentů.
- Lokální vývoj projektů s nástroji pro soubory, Git, shell, testy, obrázky a rollback.
- Ovládání aplikací ve Windows podle screenshotů.
- Trvalé projekty, více chatů v projektu, třívrstvá paměť, připnuté soubory a volitelné skills.
- Anglické a české rozhraní, které lze přepnout bez ztráty aktivního chatu.

Co model vidí

Výběr projektu dá modelu přístup k danému adresáři prostřednictvím nástrojů. Neznamená to, že se obsah všech souborů vloží do kontextového okna. Model dostane kompaktní mapu projektu a jednotlivé soubory čte podle potřeby. Výjimkou jsou připnuté soubory a tři vrstvy paměti, které jsou záměrně vkládány jako trvalý kontext.

2. Požadavky a instalace

Doporučená pracovní stanice

Součást	Doporučená konfigurace
Operační systém	Windows 11, 64 bitů
GPU	NVIDIA RTX 5090 s 32 GB VRAM
Ovladač	Aktuální NVIDIA ovladač kompatibilní s přibaleným CUDA buildem
Systémová RAM	Dost pro Windows, mapování modelu, projekty a nástroje; komfortní je 64 GB a více
Volné místo	Podle vybraných modelů; Flash-Next samostatně potřebuje přibližně 90,9 GB
Python	Full: vlastní Python 3.12 přibalený. Minimal: 64bitový Python 3.12 instalovaný samostatně
Desktopové okno	WebView2 se připraví automaticky; Full obsahuje offline runtime
Prohlížečové nástroje	Microsoft Edge, běžně součást Windows 11

Jiné NVIDIA karty mohou fungovat, ale dodané kontexty a kvantizace byly nastaveny a ověřeny pro RTX 5090 s 32 GB. Karty s menší VRAM potřebují menší kontext, nižší kvantizaci, méně GPU vrstev nebo CPU offload.

POZNÁMKA: K dispozici jsou dva instalátory. **Minimal** stahuje balíčky a llama.cpp a vyžaduje samostatně nainstalovaný Python 3.12. **Full** obsahuje vlastní Python 3.12, uzamčené balíčky a llama.cpp/CUDA knihovny. Full vytvoří nové izolované venv místně, neregistruje Python a nemění systémový PATH. Obě varianty stahují modely samostatně; ovladač NVIDIA instaluje uživatel. Offline Backup zůstává samostatným balíčkem.

WebView2 se při instalaci automaticky rozpozná a připraví. Full obsahuje celý runtime; Minimal jej stáhne jen v případě potřeby. Existující kompatibilní runtime se použije beze změny. Pokud je WebView2 později odstraněn, Marvin jej znovu připraví při spuštění. Není potřeba navštěvovat stránky Microsoftu ani jej instalovat samostatně.

Instalace pomocí Setup.exe

1. Pro soběstačné prostředí aplikace vyberte Full. Pouze pro Minimal nejprve nainstalujte 64bitový Python 3.12 a zapněte **Add Python to PATH**.
2. Spusťte Marvin-Setup-<verze>.exe.
3. Zvolte angličtinu nebo češtinu. Výchozí je angličtina.
4. Zvolte, zda chcete ikonu na ploše.
5. Při první instalaci ponechte zvolený setup prostředí a modelů po dokončení instalátoru.
6. Nechte konzolovou instalaci doběhnout. Vytvoří Python virtuální prostředí, nainstaluje závislosti, stáhne llama.cpp a nakonfigurované modely.
7. Spusťte **Marvin** z nabídky Start nebo z plochy.

Výchozí instalační adresář je:

```
%LOCALAPPDATA%\QwenHarness
```

Objem stahování závisí na zvolených modelech. Přerušené stahování lze obnovit opětovným spuštěním **Instalace prostředí a modelů** z nabídky Start. Volitelný Flash-Next se stahuje až při výběru v aplikaci.

Qwen3.8-Flash-Next

V nastavení modelu vyberte **Qwen 3.8 Flash-Next · Q3**. Marvin před stahováním zkontroluje paměť počítače, stáhne a ověří všechny části modelu včetně podpory obrázků a připraví spuštění. Během přenosu ukazuje procenta a objem dat; přenos lze zastavit a později obnovit. Ovládání chatu, příloh a nástrojů zůstává stejné.

Tento model používá Q8 cache a alespoň 128k kontext. Počet jader a rozdělení dat mezi systémovou a grafickou paměť aplikace volí sama. Pokud větší zvolený kontext nemá dost paměti, použije nižší profil, nejméně 128k, a zobrazí skutečnou velikost. Karta s menší VRAM potřebuje více volné systémové RAM. Dlouhý nový dokument se může zpracovávat několik minut; navazující otázky využívají již zpracovaný kontext.

První spuštění

Marvin po spuštění automaticky načte naposledy úspěšně používaný model s jeho profilem KV cache. Platí to i při přímém spuštění webového rozhraní. Neúspěšné přepnutí nepřepíše poslední úspěšnou volbu. Tlačítko startu nebo restartu se okamžitě rozsvítí a během načítání jemně pulzuje. Panel modelu ukazuje fázi, uplynulý čas a využití paměti grafické karty. Stop zůstává během načítání dostupný.

Nahrávání příloh, export, změny nastavení, obnova a další požadavky mají okamžitou odezvu a následné potvrzení nebo chybu. Potvrzení znamená přijetí požadavku; zařazená práce modelu dále běží v přehledu konverzace. Zálohování ukazuje průběžný výpis i dokončení či selhání v sekci Data a zálohy.

Desktopový launcher kontroluje Python prostředí, závislosti, llama.cpp a potřebné soubory modelů. Chybějící části spustí instalační workflow. Po dokončení instalace se při běžném spouštění modely znovu nestahují.

Spuštění desktopové aplikace otevře nativní WebView okno, spustí Web UI a podle potřeby aktivuje llama-server s vybraným modelem. Zavření desktopové aplikace zastaví její Web UI i modelový server a uvolní grafickou paměť.

Aktualizace

Nainstalujte nový Setup.exe přes existující instalaci. Uživatelská data, modely, relace, projekty, uživatelské skills a uložený stav UI zůstanou zachovány. Soubory aplikace a distribuované systémové skills se aktualizují. Model se znovu stahuje jen tehdy, když nakonfigurovaný soubor chybí.

Odinstalace a odstranění všech dat

Odinstalace ve Windows odstraní nainstalované soubory aplikace. Velká runtime data a uživatelská data mohou zůstat, aby aktualizace nebo reinstalace nemusela vše znovu stahovat.

Chcete-li odstranit úplně vše, zkontrolujte a smažte zbývající instalační adresář pouze tehdy, když už nepotřebujete `runtime\models`, `sessions`, `projects`, `memory`, `user-skills` a `projects.json`.

VAROVÁNÍ: Smazání těchto adresářů je nevratné. Nejdříve zálohujte projekty, relace, paměť a uživatelské skills.

Spuštění ze zdrojového checkoutu

```
py -3.12 -m venv .venv
.venv\Scripts\python scripts\setup_env.py --model auto
npm --prefix frontend ci
npm --prefix frontend run build
.venv\Scripts\python qwen_app.py
```

Pro terminál použijte `run_cli.bat` nebo `.venv\Scripts\python tui.py`.

Při práci ze zdrojů je potřeba Node.js pro sestavení frontendu. Běžná instalace Setup.exe už sestavený frontend obsahuje a Node.js nevyžaduje.

3. Prohlídka desktopového rozhraní

Vlevo je navigace projektů a chatů, uprostřed konverzace a vpravo zavíratelný panel detailů. Prompt zůstává dole pod konverzací. V menším okně se detail otevírá jako zásuvka.

Tato kapitola popisuje pracovní okno verze 1.8.0. Model a jeho stav najdete v horní liště. Pravý sloupec otevřete ikonou detailu nad konverzací nebo kliknutím na ukazatel kontextu pod zprávou. Obsah pravého sloupce se posouvá samostatně; jeho spodní patička © **Petr Sajner 2026** zůstává na místě. Stejný copyright je ve startovacím okně.

První úloha krok za krokem

1. Klikněte na název modelu nahoře a zkontrolujte volbu v **Model a zařízení**. Pro běžnou práci ponechte automatickou detekci paměti. Počkejte na stav **Připraven**.
2. Vlevo vyberte projekt, připojte existující složku přes menu projektu nebo ponechte **Bez projektu**. Klikněte na **Nový chat**.
3. Vedle názvu chatu zvolte pracovní režim. Například **Psaní** pro dokument nebo **Vývoj** pro práci v repozitáři.
4. Napište požadovaný výsledek. Podle potřeby přidejte podklady tlačítkem **Attach**, přetažením nebo vložením ze schránky.
5. Odešlete zprávu. Průběh sledujte v konverzaci a v pravé záložce **Průběh**. Během práce lze upřesnit zadání, připravit další zprávu do fronty nebo úlohu zastavit.
6. Hotové soubory najdete v **Výsledky**. Otevřete náhled nebo jejich složku a ověřte výsledek. Běžná textová odpověď zůstává přímo v chatu.

Navigace a pracovní režimy

Šířku navigace i pravého detailu změníte tažením za jejich vnitřní okraj. Dvojklik na okraj obnoví výchozí šířku; totéž provede restart nebo obnovení stránky. Seznam konverzací nejprve ukáže 20 nejnovějších chatů vybraného projektu. **Zobrazit starší** přidá dalších 20, **Jen nejnovější** seznam opět zkrátí.

Desktopový spouštěč používá data své vlastní instalace. Vývojová kopie a nainstalovaný Marvin neslučují konverzace. Pro výslovné otevření existujících dat z vývojové kopie slouží `python webapp.py --data-dir "C:\cesta\k\Marvinu"`. Data zůstávají na původním místě, bez převodu nebo kopírování.

Výběr projektu otevře jeho nejnovější chat. **Bez projektu** otevře samostatnou konverzaci. Přepnutí pohledu nepřesune ani nepřenastaví úlohu, která právě běží v jiném chatu.

Režimy mají vždy pořadí **Diskuze, Výzkum, Psaní, Vývoj, Počítač**. Režim patří konkrétnímu chatu. Probíhající požadavek si zachová konfiguraci, se kterou začal.

Tlačítko **Nový chat** vytvoří konverzaci. Název nad chatem přejmenujete zadáním textu a Enterem. Třítečkové menu chatu obsahuje přesun, vrácení posledního kola, větev, export a smazání. Výběr cílového projektu chat přesune a přepočítá jeho projektový kontext. Před přesunem nebo smazáním zastavte úlohu tohoto chatu.

Menu vedle výběru projektu vytvoří nový projekt, připojí existující složku nebo smaže projekt i jeho adresář. Vyhledávání vlevo prohledává uložené konverzace.

Prompt, Attach a náhledy obrázků

Viditelné tlačítko **Attach** zůstává dole u promptu. Otevře výběr z disku a dovoluje vybrat více souborů. Obrázky lze také přetáhnout do pracovní plochy nebo vložit ze schránky přes Ctrl+V. Běžné vložení textu funguje dál.

Každý připravený obrázek má skutečný náhled a vlastní křížek pro odebrání. Odebráním jednoho obrázku nezmizí text ani ostatní přílohy. Kliknutí zvětší originál. Po odeslání jsou náhledy součástí uživatelské zprávy a zůstanou dostupné i po opětovném otevření chatu.

Přiložit lze také PDF, DOCX, XLSX, CSV, Markdown a text. Model dostane cesty k dokumentům a přečte je příslušnými nástroji. Textový model neumí obsah obrázku; pro obrazové vstupy zvolte model s vision. Při odmítnutém požadavku zůstanou text i přílohy v draftu.

Enter a Ctrl+Enter odešlou zprávu, Shift+Enter vloží nový řádek. Composer se vyčistí po přijetí identifikované zprávy službou. Rozepsaný text se ukládá pro každý chat zvlášť.

Běžící úloha a fronta zpráv

Během práce zvolte **Upřesnit nyní** pro steering nebo **Po dokončení** pro samostatný další požadavek. Úloha v jiném chatu pokračuje a nová zpráva čeká na jediný modelový worker. Text ve frontě lze upravit a čekající zprávu zrušit.

Zastavit úlohu funguje při zpracování promptu, přemýšlení, psaní, přípravě nástroje i synchronní akci. U viditelného textu může krátce dokončit větu; nečeká na dokončení celého reasoning bloku. Hotová práce i částečný text zůstanou zachované. Stop pozastaví také čekající frontu; **Spustit zprávy ve frontě** ji opět spustí. **Pokračovat** obnoví přerušenu úlohu s právě vybraným modelem a jeho aktuálním KV profilem. Po ručním přepnutí se již nenačítá původní model úlohy. Historie a rozpracovaná úloha zůstávají zachované; menší kontext používá běžnou kompresi historie.

Reload a obnovení spojení prohlížeče neruší běžící modelovou úlohu. Po restartu celé aplikace lze přerušenu práci obnovit. Když před pádem nebyl uložen výsledek nástroje, model dostane informaci, aby ověřil skutečný stav místo slepého opakování akce.

Výsledky, Průběh a Kontext

Výsledky ukazují vytvořené dokumenty a změněné soubory, skutečně zaznamenané kontroly, výzkumné exporty a body obnovy. Soubor můžete prohlédnout, otevřít jeho složku nebo stáhnout. HTML výsledek lze zobrazit jako aplikaci a podporované programy mají také akci **Spustit**.

Průběh spojuje plán, trvale uložené provozní události, změny souborů, procesy na pozadí s výstupem a stav izolovaného browseru. Samostatné procesy podle potřeby zastavujete zvlášť.

Kontext zobrazuje odhad obsazení a fyzický limit, naměřenou spotřebu posledního dokončeného požadavku, pokud je dostupná, tři paměťové vrstvy, pins a načtené skilly. Můžete připnout či odepnout jednotlivé soubory, zrušit všechny pins, komprimovat kontext nebo předat práci novému chatu.

Při generování stavový řádek rozlišuje přípravu, uvažování, psaní odpovědi, přípravu nástroje a provádění. Ukazuje také uplynulý čas a výslovně přibližný počet právě generovaných tokenů.

Nastavení

Klikněte na ikonu nastavení nebo stav modelu nahoře. Nastavení je rozdělené do kategorií:

- **Model a zařízení:** model, KV profil, vision, paměť GPU, provozní diagnostika a Start / Stop / Restart.
- **Chování:** autonomie a výchozí zacházení se zprávou přidanou během práce.
- **Paměť a skilly:** úprava globální, režimové a projektové paměti; čtení, použití a návrh skillu; otevření uživatelské i projektové složky skillů.
- **Data a zálohy:** export/import celého projektu, import JSONL chatu, vytvoření/výběr/kontrola lokální zálohy prostředí a stav údržby.
- **Vzhled a jazyk:** tmavý, světlý nebo systémový vzhled, rozestupy, angličtina a čeština.
- **Nápověda a manuály:** oba PDF manuály a reference lomítkových příkazů.

Myšlení zůstává také u promptu, protože ho můžete měnit mezi otázkami. Nastavení zvolené za běhu platí pro následující požadavky. Přepnutí jazyka a vzhledu nezahazuje konverzaci.

Projektová rozhodnutí

Přijatá rozhodnutí evidují navržená, přijatá i neplatná rozhodnutí. Rozhodnutí lze přidat, upravit, změnit jeho stav a otevřít původní konverzaci. Do projektového kontextu automaticky vstupují jen přijaté položky. Doplňují tři paměti; návrh se sám nemění na závazný fakt.

4. Modely, KV cache, kontext a thinking

Model	Typické použití	Kvantizace	KV	Praktické profily
Qwen 3.8 27B Q5	Hlavní kvalitní model	Q5	F16/Q8	F16 96k; Q8 192k
Qwen 3.8 27B Q4	Rychlost a největší kontext	Q4	F16/Q8	F16 128k; Q8 256k
Qwen 3.8 27B IQ3_S	Menší grafické karty, kompromis v kvalitě	IQ3_S	F16/Q8	Podle GPU: Q8 32k až 256k
Ornith 1.5 35B-A3B Abliterated Q5	Velmi rychlý volitelný MoE	Q5	Q8	128k
Nemotron 3.5 Lightning Q4	Rychlý textový MoE	Q4_K_XL	Q8	128k, 256k, 512k; také profily s využitím RAM
Nemotron 3.5 Lightning Q5	Textový MoE s vyšší kvantizací vah	Q5_K_XL	Q8	32k, 64k, 128k; 256k s využitím RAM
Qwen 3.8 Flash-Next Q3	Velký model s automatickým využitím GPU a RAM, včetně vision	Q3_K_XL	Q8	128k, 192k, 256k; výchozí požadavek 256k

Výchozí profil nové instalace je Qwen Q5/Q8/192k. Aplikace si pamatuje poslední model a KV volbu každého modelu.

Q5 používejte pro náročný vývoj, architekturu a finální kvalitu. Q4 je vhodný pro vyšší rychlost nebo kontext 256k. Ornith je extrémně rychlý, ale při reálném vývoji může být slabší než dense Qwen.

Nemotron je pouze textový. Qwen, Ornith a Flash-Next mají vlastní podporu obrazových vstupů. Nabídka modelů ukazuje dostupnost souborů; u Flash-Next musí být kompletní všechny shardy i projektor. Podle karty jsou u Qwenu dostupné také kompaktní profily. Tabulka není příslibem, že každá kombinace poběží na každém počítači; přesný výběr najdete u zvoleného modelu v nastavení.

Flash-Next: nastavení a reálné čekání

Flash-Next stáhne přibližně **90,9 GB** ve čtyřech souborech. Po prvním stažení se používá místní kopie. Změna KV profilu model znovu nestahuje. Během přípravy uvidíte zvlášť stahování, kontrolu souborů a načítání; přenos ukazuje procenta a objem dat.

Výchozí požadavek je **256k Q8**. Aplikace podle aktuálně volné RAM a VRAM vybere 256k, 192k nebo 128k, nastaví CPU a rozdělení vah a zobrazí skutečně použitý profil. Pod 128k tento model nenastavuje. Když paměť nestačí, vysvětlí problém a při neúspěšném přepnutí se pokusí obnovit předchozí funkční model.

Na RTX 5090 / 64 GB RAM / Core Ultra 7 265K jsme naměřili přibližně **27 generovaných tokenů za sekundu**. Úplné zpracování nového vstupu o 122 397 tokenech trvalo **17 minut 34 sekund**. Následující otázka využila uložený kontext a odpověď trvala **1,27 sekundy**. Rychlost psaní odpovědi proto neříká, jak dlouho potrvá načíst rozsáhlý nový dokument.

128k profil prošel tímto dlouhým testem. 256k prošel testy nástrojů, obrázků, agentní úlohy a vstupu o 24 tisících tokenech; celé 256k okno nebylo naplněno. Na fyzických 16GB a 24GB kartách tato varianta zatím změřená není. Menší VRAM vyžaduje více volné systémové RAM; samotný údaj o celkové RAM nestačí.

Profily používají tradiční označení 128k/192k/256k pro 131 072 / 196 608 / 262 144 tokenů. Číselný ukazatel může tutéž kapacitu zaokrouhlit například na 262k.

Přesnost KV

Kvantizace vah a KV cache jsou dvě různé věci: **Q3** u Flash-Next popisuje váhy modelu, **Q8** jeho průběžnou paměť kontextu. F16 používá více paměti pro KV, Q8 ji šetří. Výslednou velikost okna určuje konkrétní profil a dostupná paměť. Změna KV restartuje server, nikoli chat.

Thinking

U Qwenu jsou xhigh, medium, low nativní reasoning effort a off thinking vypíná. U Ornithu je nativní pouze on/off: xhigh a medium jsou promptové vodítko, low běžné thinking chování a off vypnutí.

Reasoning tokeny používají čas i kontext; UI je živě odhaduje. Uložené uvažování lze rozbalit u odpovědi. Odhad persistentního kontextu zahrnuje i uvažování předávané zpět modelu.

Produkční agent nemá limit kroků ani aplikační limit výstupních tokenů.

5. Odesílání, steering, Stop a průběh

Prompt a přílohy

Pište přirozeným jazykem a uveďte požadovaný výsledek, omezení a způsob ověření. Pomocí **Attach**, přetažení nebo Ctrl+V lze přidat BMP, GIF, JPEG, PNG nebo WebP: fotografie, diagramy, screenshoty, UI reference i chyby.

Steering

Při volbě **Upřesnit nyní** další zpráva v právě běžícím chatu přesměruje aktivní úlohu. Hotový text zůstane uložen a práce pokračuje s upřesněním. **Po dokončení** vytvoří samostatný následující požadavek. Zpráva do jiného chatu čeká ve frontě na jediný model. Zpráva doručená těsně při dokončování se neztratí a přejde do čekající práce.

Stop

Stop obchází běžnou frontu a měkce ukončí generování po nejbližší větě. Současně zruší právě čekající browser operaci nebo synchronní run_command a ukončí jeho procesní strom. Samostatný proces spuštěný na pozadí zastavte zvlášť v **Průběh > Procesy**.

Rozlišujte **Zastavit úlohu** u zprávy a **stop** v nastavení modelu. První ukončí aktuální práci a pozastaví frontu, ale model může zůstat připravený v paměti. Druhé zastaví modelový server a uvolní jeho prostředky. Stažené modely zůstávají na disku pro další spuštění.

Živý průběh

UI rozlišuje reasoning, viditelný text, přípravu velkých tool callů, provádění nástrojů/příkazů/testů a sekundy bez tokenů. Při tvorbě velkého souboru ukazuje jméno souboru a rostoucí množství připravovaného obsahu.

6. Pracovní režimy

Režim	Použití
Diskuze	Běžný chat, analýza, učení a brainstorming bez coding procedur.
Výzkum	Web/dokumentové rešerše, persistentní ledger a závěrečná syntéza.
Psaní	Scénáře, články, reporty, textové revize a export dokumentů.
Vývoj	Repozitář, soubory, Git, shell, testy, checkpoint a rollback.
Počítač	Vývoj plus screenshot, myš, klávesnice a scrolling.

Každý chat si pamatuje vlastní režim. Režimy jsou profily schopností stejného modelu, ne samostatní agenti.

7. Projekty a workspace

Vytvoření a připojení

V třítečkovém menu vedle projektu vytvořte spravovaný projekt nebo pomocí **Připojit existující složku** zaregistrujte adresář bez kopírování.

Projekt může mít libovolný počet chatů. Každý má vlastní historii, režim, kompresi, pins, research a task state; sdílí složku a projektovou paměť.

Bez projektu a přesun chatu

Bez projektu používá samostatný chat; exporty jdou do relace. V třítečkovém menu chatu zvolte cílový projekt. Přesun přenastaví workspace, projektovou paměť a knihovnu dokumentů.

Smazání projektu

Klikněte **Smazat projekt i složku**, ověřte zobrazenou cestu a potvrďte v dialogu. Smaže se registrace, všechny chaty projektu, složka i veškerý obsah.

VAROVÁNÍ: Potvrzením se smaže i připojená externí složka. Nejde o odpojení. Kritické systémové cesty jsou blokovány, ale cestu vždy zkontrolujte.

8. Správa chatů

- **Nový chat** vytvoří transientní relaci, která se uloží po první zprávě.
- **Smazat chat** vyžaduje potvrzení v dialogu a odstraní relaci včetně příloh, research a session exportů; projektové soubory zůstanou.
- Přejmenování se potvrzuje Enterem.
- **Znovu** zopakuje odpověď na poslední prompt.
- **Vrátit** odstraní poslední konverzační kolo, nikoli souborové změny.
- **Větev** vytvoří kopii chatu u posledního promptu včetně projektu, režimu, obrázků a pins.
- **Hledat ve všech chatech** používá SQLite fulltextový index.
- Export podporuje čitelný Markdown a strukturální JSONL; JSONL se importuje jako nový chat.
- **Předat chatu** vytvoří nový chat se souhrnem cílů, rozhodnutí, stavu a dalších kroků.

9. Kontext, komprese a pins

Panel **Kontext** ukazuje odhad tokenů, viditelné/celkové zprávy, obrázky, stav komprese a pins. Rozpad zvlášť uvádí konverzaci/přílohy, aktuální projektový kontext a definice nástrojů. Celkový údaj proto zahrnuje i režii tool schemas.

Ve Vývoji model dostává kompaktní repo snapshot; ostatní režimy katalog dokumentů. Snapshot, pins a aktivní projektové instrukce se skládají jen pro aktuální request a neukládají se jako stále nové kopie do historie.

Harness automaticky hledá AGENTS.md, QWEN.md a CLAUDE.md od kořene projektu po adresář právě čteného nebo měněného souboru. Hlubší dokument je konkrétnější. Jde o guidance; aktuální zadání uživatele má přednost.

Připnout soubor vloží text vybraného souboru do následujících úloh tohoto chatu. Vhodné jsou architektura, specifikace, handoff a projektová pravidla. Limit je 10 souborů a asi 40 000 znaků. Větev pins kopíruje, nový chat ne.

Při 85 % kontextu se starší část automaticky shrne. Viditelná historie se nemaže. Ruční komprese je v **Kontext > Komprimovat**. Při overflow agent jednou automaticky komprimuje a request zopakuje.

10. Tři vrstvy paměti

Vrstva	Rozsah	Umístění
Globální	Všechny režimy/projekty	memory\GLOBAL.md
Režimová	Všechny chaty daného režimu	memory\MEMORY.md nebo memory\modes*.md
Projektová	Jeden workspace	<projekt>\QWEN_MEMORY.md

Paměti otevřete v **Nastavení > Paměť a skilly** nebo řekněte modelu "zapamatuj si globálně", "pro Výzkum" či "pro tento projekt". Přesun chatu přepne projektovou paměť.

11. Volitelné skills

Skills jsou pomocné SKILL.md postupy. Nespouštějí druhý model a nepřebíjejí explicitní zadání. Model vidí jen jméno a trigger popis; celé tělo načte přes read_skill podle potřeby.

Priorita je projektový .qwen-skills, potom user-skills, potom distribuované skills.

Skill	Účel
systematic-debugging	Reprodukce, trasování, hypotézy a příčina chyby.
architecture-options	Varianty architektury se zachováním zadání.
implementation-verification	Přiměřená kontrola výsledku, testů a releasu.
performance-investigation	Měřením řízená analýza výkonu.
research-synthesis	Úplná syntéza se zachováním rozporů.

Vlastní skill vytvořte přes **Nastavení > Paměť a skilly > Uživatelské skilly** jako user-skills\nazev\SKILL.md:

```
---
name: muj-skill
description: S čím pomáhá a kdy ho má model načíst.
---

# Můj skill

Flexibilní doporučení a reference.
```

12. Internet a výzkum

Běžná práce s webem

Všechny režimy mohou používat `web_search` a `web_fetch` pro aktuální informace, veřejnou dokumentaci, chybové zprávy a webové stránky. `web_fetch` zpracuje HTML a umí extrahovat text z podporovaných PDF a DOCX downloadů.

U dlouhých stránek může model vyhledat konkrétní výraz nebo načíst další pasáž bez opakovaného stahování. Celý načtený zdroj zůstává v evidenci výzkumu v rámci jejího technického limitu; užší výřez jej nenahrazuje. To pomáhá zvláště u velkých modelů, které část výpočtu provádějí v systémové RAM.

Během načítání vstupu se zobrazuje **Načítám kontext**, průběh nového textu a počet tokenů použitých z cache. Jde o jinou fázi než generování přemýšlení a odpovědi. První dlouhý dokument může být pomalý, i když navazující otázka využije téměř celou historii z cache. Úroveň přemýšlení a zvolená kvantizace se tímto zrychlením nemění.

Výchozí vyhledávač je Google; v `config.yaml` lze zvolit Google, Bing nebo automatický fallback. Fetching je read-only HTTP/HTTPS.

Výzkumný workflow

Režim Výzkum automaticky:

1. Zaznamená otázku.
2. Před hledáním vytvoří trvalý research plán.
3. Uloží každý dotaz a kandidátní odkaz.
4. Uloží plný čitelný obsah webových i lokálních zdrojů.
5. Sleduje coverage a ID zdrojů.
6. Zachová rozpory a menšinové informace.
7. Vytvoří závěrečnou lidsky čitelnou syntézu.

Zdroje se neskrývají ani nevyhazují podle toho, zda je model považuje za důvěryhodné. Důvěryhodnost a relevanci posuzuje uživatel. Syntéza odlišuje tvrzení zdrojů od inference a uvádí nejistoty a chybějící důkazy.

Panel výzkumu

V pravé záložce **Výsledky** otevřete sekci **Výzkum**. **Všechny zdroje** zobrazí podklady aktuální úlohy; tlačítka **PDF**, **DOCX** a **Zdroje** exportují hotovou syntézu nebo její zdrojový záznam. Obecný plán a provozní události jsou v záložce **Průběh**. Export již hotové odpovědi nespouští nový výzkum.

Podporované projektové dokumenty zahrnují Markdown, text, RST, CSV, JSON, YAML, PDF a DOCX. Když Výzkum přečte lokální dokument, zaznamená jej jako zdroj.

13. Psaní a export dokumentů

Režim Psaní zachovává záměr, hlas, strukturu a omezení uživatele. Umí číst a upravovat projektové soubory bez zavádění vývojové terminologie, pokud ji uživatel nepožaduje.

Model může v každém režimu exportovat finální text přirozeným požadavkem:

```
Ulož finální odpověď jako PDF.
Vyexportuj to jako strukturovaný DOCX s názvem production-report.
Vytvoř Markdown soubor s touto syntézou.
```

Podporovány jsou Markdown, strukturovaný DOCX a PDF s Unicode/českým textem, základními tabulkami a inline formátováním.

S projektem se výstup ukládá do <projekt>\exports. Bez projektu jde do sessions\<id-chatu>\exports. Projektové exporty mohou být součástí task checkpointu a lze je podle situace rollbackovat.

14. Vývojový workflow

Poznání projektu

Při každém requestu dostane model aktuální repo snapshot a platné hierarchické projektové instrukce. Může použít repo_overview, project_instructions, procházet/globovat soubory, hledat literal nebo regex, navigovat deklarace přes find_symbol/document_symbols, hledat použití přes find_references a číst relevantní rozsahy.

Souborové operace

- read_file čte text s čísly řádků.
- search_files používá ripgrep, podporuje literal/regex, case sensitivity a glob názvu.
- find_files vrací soubory odpovídající globu.
- write_file vytvoří nebo přepíše celý soubor.
- apply_patch provádí přesné atomické náhrady.
- make_directory, move_file, delete_file jsou strukturované operace zahrnuté do rollbacku.
- view_image vloží obrázek do vision kontextu.

Izolovaný browser workflow

Vývoj a Počítač obsahují persistentní headless Microsoft Edge session oddělenou od běžného browser okna uživatele:

1. browser_open načte lokální nebo veřejnou URL.
2. browser_snapshot vrátí viditelný text a interaktivní prvky s refs jako e1.
3. browser_fill, browser_click, browser_press pracují přes tyto refs.
4. browser_select, browser_hover, browser_scroll obslouží dropdowny, hover a dlouhé stránky.
5. browser_upload nastaví lokální file input; browser_download uloží download k aktivnímu chatu.
6. browser_viewport přepne desktop, tablet nebo mobilní rozměr.
7. Nový snapshot ověří DOM výsledek; zastaralé refs jsou odmítnuty.
8. browser_console a browser_network vracejí konzoli, HTTP responses a selhané requesty.
9. browser_screenshot vloží vykreslenou stránku do dalšího requestu pro Qwen vision.
10. browser_close nebo **Browser session > Zavřít browser** uvolní Edge proces.

Browser přežije jednotlivé agentní kroky. Režim Počítač použijte pro nativní desktopové aplikace nebo celou obrazovku Windows.

Plán úlohy

U větší práce model vytvoří operační plán pomocí set_task_plan a aktualizuje kroky přes update_task_step. Panel **Průběh úlohy** ukazuje cíl, pending/in-progress/completed kroky, poslední validaci a kontrolu diffu. Ledger je v task-plan.json, přežije restart i kompresi a neobsahuje soukromý chain-of-thought.

Checkpoint a rollback

První zápis aktivuje change journal. **Změny této úlohy** vypíší vytvořené/upravené soubory. **Vrátit změny této úlohy** obnoví journalované soubory do stavu před úlohou a odstraní soubory, které úloha vytvořila.

Chatové **Vrátit** mění konverzaci. Task rollback mění souborový systém. Jde o dvě odlišné funkce.

Git

Model umí číst status a diff a vytvořit lokální commit. `git_commit` staguje uvedené cesty nebo jen soubory aktuálního task journalu. Automaticky nepushuje. Commit i push požadujte výslovně.

Příkazy a testy

Krátké příkazy běží synchronně s timeoutem. Úplný stdout/stderr se ukládá do `sessions\<id-chatu>\command-logs`; model dostane začátek i konec, takže neztratí závěrečnou chybu. Stop aktivní synchronní příkaz ihned ukončí. Dlouhé příkazy na pozadí vrátí process ID a lze je pollovat, posílat jim stdin nebo ukončit celý strom procesů.

`project_validation_profile` vypíše detekované test/lint/typecheck/build příkazy. `start_project_check` spustí primární nebo pojmenovanou kontrolu. Detekce pokrývá tento harness, pytest, Node scripts, Rust, Go a .NET.

Projekt může detekci nahradit souborem `.qwen/project.yaml`:

```
checks:
- id: tests
  label: Kompletní testy
  command: npm test
  shell: powershell
  kind: test
  timeout: 900
  primary: true
```

Dokončená kontrola se zapíše do panelu. Pokud změněná úloha končí bez validace, dokončeného plánu nebo kontroly diffu, harness model jednou upozorní. Model užitečnou kontrolu provede, nebo vysvětlí, proč pro danou úlohu nemá smysl.

Finální kontrola je doporučení, ne omezení. Explicitní zadání uživatele na architekturu, formu, rozsah nebo single-file výsledek má přednost.

15. Ovládání počítače

Počítač přidává GUI nástroje ke kompletní sadě Vývoje.

Cyklus

1. Model zavolá screenshot.
2. Obrázek se podle potřeby zmenší a přiloží.
3. Model určí prvky v souřadnicích obrázku.
4. Klikne, pohne myší, scrolluje, píše nebo stiskne klávesy.
5. Dalším screenshotem ověří výsledek.

Harness přepočítá souřadnice obrázku na skutečné rozlišení primárního displeje.

GUI akce

- Screenshot a informace o obrazovce.
- Pohyb myši pro hover.
- Levé, pravé, prostřední kliknutí a dvojklik.
- Scroll na volitelné pozici.
- Přímé psaní ASCII nebo vložení Unicode/dlouhého textu přes schránku.
- Klávesy a kombinace jako Enter, Escape, Ctrl+S, Alt+F4 nebo Win+D.

Failsafe

Rychlým přesunutím fyzického kurzoru do levého horního rohu primární obrazovky aktivujete PyAutoGUI failsafe a přerušíte GUI akce.

VAROVÁNÍ: Počítač vidí obrazovku a může provádět destruktivní akce. Pro e-mail, platby, účty, mazání a citlivé workflow používejte Supervised autonomii.

16. Autonomie a potvrzování

Úroveň	Chování
Supervised	Každý zápis, shell akce a GUI akce vyžaduje potvrzení; čtení pokračuje.
Semi	První WRITE akce v úloze vyžádá potvrzení; potom úloha pokračuje.
Auto	WRITE akce pokračují bez potvrzení.

V produkci není limit agentních kroků.

Při potvrzení chat vypíše čekající akce a zobrazí **Povolit** a **Zamítnout**. Zamítnutí se vrátí modelu, aby mohl zvolit jiný postup.

Přesměrování, instalace balíčků, libovolný Python, kopírování, mazání, síťové zápisy, Git commit/push a smíšené příkazy se konzervativně považují za WRITE.

17. Terminálové UI a CLI

Instalátor přidá **Marvin (CLI)** do nabídky Start. Spouští `run_cli.bat` a interaktivní TUI.

Příkaz	Účel
<code>/memory</code>	Zobrazit globální, režimovou a projektovou paměť.
<code>/model q4 q5 ornith_q5</code>	Přepnout model a restartovat server.
<code>/work discussion research writing development computer</code>	Zvolit pracovní režim.
<code>/mode chat agent computer</code>	Legacy kompatibilní režimová zkratka.
<code>/autonomy supervised semi auto</code>	Nastavit potvrzování.
<code>/thinking xhigh medium low off</code>	Nastavit hloubku uvažování.
<code>/ws [cesta]</code>	Zobrazit nebo nastavit workspace.
<code>/img <cesta></code>	Přiložit obrázek k dalšímu promptu.
<code>/screenshot</code>	Zachytit obrazovku.
<code>/new, /sessions, /load <id></code>	Spravovat relace.
<code>/server status start stop</code>	Ovládat inference server.
<code>/help, /exit</code>	Nápověda a ukončení.

Při potvrzení y povolí, n zamítne a a povolí zbývající WRITE akce v úloze. Ctrl+C přeruší generování.

18. Úložiště, zálohy a logy

Cesta	Obsah
runtime\models	GGUF modely a vision projektory
runtime\llama	CUDA binárky llama.cpp
runtime\workspace-settings.json	Aktuální nastavení webového pracovního okna a poslední úspěšný model
runtime\application.sqlite3	Fronta úloh, trvalé události a registr souborů
runtime\execution-plans	Automaticky vypočtené nastavení běhu Flash-Next
sessions\<id>	Zprávy, přílohy, task state, research, komprese a exporty
projects	Projekty vytvořené aplikací
projects.json	Registr projektů
memory	Globální a režimová paměť
<projekt>\QWEN_MEMORY.md	Projektová paměť
skills, user-skills, <projekt>\.qwen-skills	Tři vrstvy skillů

Zálohujte projektové adresáře, sessions, memory, user-skills, projects.json a stavové soubory v runtime. Starší webui-state.json slouží pro kompatibilitu a převzetí dřívějších voleb; hlavní UI používá workspace-settings.json.

Kompletní offline záloha instalace

Aplikace umí vytvořit přenosnou instalační zálohu přímo ze souborů, které už jsou na tomto počítači. Otevřete **Nastavení > Data a zálohy**, zvolte **Vytvořit zálohu** a vyberte nadřazený adresář, ideálně na jiném disku. Vznikne časově označená složka QwenHarness-Offline-Backup-*, která obsahuje:

- Všechny kompletní modely a vision projektory z runtime\models.
- Nainstalované llama.cpp a CUDA runtime z runtime\llama.
- Aktuální výběr modelů a requirements.txt.
- Kopii Python balíčků, které už jsou nainstalované v .venv.
- Odpovídající Setup.exe, pokud je dostupný v aktuálním build adresáři; verze 1.8 přednostně přikládá Full s vlastním Pythonem.
- manifest.json s velikostí a SHA-256 hashem každého zálohovaného souboru.

Modely, runtime i Python závislosti se kopírují přímo z aktuálního adresáře Marvin. Při tvorbě zálohy se nic z toho znovu nestahuje. Je potřeba přibližně tolik volného místa, kolik zabírá nainstalovaný runtime; operace může trvat, protože se každý soubor kontrolně hashujete.

Instalace z offline zálohy

1. Zkopírujte celou offline složku, včetně manifest.json, payload, python-dependencies a příloženého instalátoru. Kompletní balíček se všemi současnými modely potřebuje přes 200 GB místa.
2. Spustěte příložený **Full** instalátor. Obsahuje vlastní Python a nevyžaduje jeho samostatnou instalaci. Jen pokud používáte **Minimal**, musíte mít 64bitový Python 3.12 připravený předem.
3. Použijte jednu z možností:
 - Pokud je Setup.exe uvnitř zálohy vedle manifest.json, spustěte ho přímo tam; zálohu rozpozná automaticky.
 - Položte zálohu vedle Setup.exe a přejmenujte ji přesně na QwenHarness-Offline-Backup; instalátor ji rozpozná automaticky.
 - Nainstalujte Marvin, v nabídce Start spustěte **Instalace z offline zálohy** a vyberte složku zálohy.

4. Když instalátor rozpozná zálohu vedle sebe, první příprava použije místní obnovu přednostně. Úplná obnova přenesne všechny modely obsažené v balíčku, včetně Flash-Next, a kontroluje jejich SHA-256. Výběr modelů v průvodci řídí případné další stahování.
5. Příkaz **Instalace z offline zálohy** v nabídce Start také obnovuje místní data přednostně. Běžné nastavení bez této volby používá internet a zaregistrovanou zálohu jako náhradní zdroj. Pokud součást není ani místně, dokončení vyžaduje její získání.

Tlačítkem **Použit jako zálohu** zaregistrujete složku pro případ budoucího selhání downloadu, **Ověřit SHA-256** zkontroluje každý soubor podle manifestu a **Zapomenout výběr** tuto pojistku vypne. Výběr zálohy nezakazuje přístup k internetu. `Setup.exe`, `manifest.json`, `README-OFFLINE.txt`, `requirements.txt`, `python-dependencies` a `payload` musí zůstat společně v jedné záložní složce.

Offline instalační záloha neobsahuje chaty, projekty, paměť ani osobní skills. Tato nenahraditelná uživatelská data zálohujte samostatně podle seznamu výše.

Log	Použití
<code>runtime\launcher.log</code>	Launcher a start aplikace
<code>runtime\app.log</code>	Nativní aplikace a crashe
<code>runtime\webapp.log</code>	Web UI a Python chyby
<code>runtime\llama-server.log</code>	Model, kontext, CUDA, inference a timing

19. Řešení problémů

Server nestartuje

Otevřete **Nastavení > Model a zařízení**, přečtěte hlášení a zkuste restart. Ověřte kompletní model, kompatibilní NVIDIA ovladač a volnou paměť. Flash-Next potřebuje vedle VRAM také dost systémové RAM. Podrobnosti jsou v `runtime\llama-server.log`.

Chybí model nebo projektor

Spusťte **Instalace prostředí a modelů** z nabídky Start. Hotové soubory se přeskočí a podporovaný nedokončený download se obnoví.

Port Web UI je obsazený

Launcher najde blízký volný port nebo znovu použije existující Marvin Web UI. URL je v `launcher.log`.

Dlouhý chat je pomalý nebo přetekl

Zkontrolujte kontext, použijte Q5/Q8 192k nebo Q4/Q8 256k, ručně komprimujte, předejte do nového chatu nebo odstraňte nepotřebné pins/obrázky. Agent při overflow jednou automaticky komprimuje a zopakuje request.

UI vypadá zamrzle

Sledujte fázi a uplynulý čas: může probíhat stahování, kontrola vah, zpracování dlouhého vstupu, thinking nebo nástroj. U Flash-Next může nový velký vstup trvat několik minut. **Zastavit úlohu** ukončí aktuální požadavek; samostatný proces na pozadí zastavte v **Průběh > Procesy**. Modelový server se zastavuje zvlášť v **Model a zařízení**.

Izolovaný browser nestartuje

- Microsoft Edge musí být nainstalován ve standardním umístění Windows.
- Po upgradu spusťte opravu/aktualizaci prostředí, aby se nainstaloval Python Playwright.
- Session je headless a nepoužívá běžné Edge/Chrome okno uživatele.

Obnovení po restartu

V **Rozpracovaná úloha** zvolte **Pokračovat v úloze**. Podle možnosti se obnoví pending potvrzení, částečný text, journal a procesní metadata.

Kde je PDF/DOCX

S projektem v <projekt>\exports, bez projektu v sessions\<id>\exports. Exportovaný soubor najdete také v pravé záložce **Výsledky**.

Jazyk se nezměnil celý

Použijte **Nastavení > Vzhled a jazyk**. UI se přepne přímo a zachová chat. Nativní launcher může vyžadovat restart. Uložená UI volba má přednost před jazykem instalátoru.

20. Reference nástrojů

Nástroje běžně požadujete přirozeným jazykem.

Všechny režimy

Nástroj	Schopnost
read_memory, save_memory	Číst a ukládat tři vrstvy paměti.
web_search, web_fetch	Hledat a číst veřejný web/dokumenty.
context_status	Tokeny, zprávy, obrázky, pins a komprese.
pin_context_file, unpin_context_file	Spravovat pins aktuálního chatu.
list_project_documents, read_project_document	Katalog a čtení dokumentů.
list_skills, read_skill	Katalog a načtení skillů.
export_document	Export Markdown, DOCX nebo PDF.
list_dir, read_file	Procházet adresáře a číst rozsahy textu.
search_files, find_files	Rychlé literal/regex hledání a glob souborů.
write_file, apply_patch	Vytvářet a atomicky upravovat text.
make_directory, move_file, delete_file	Strukturované změny s rollbackem.
list_task_changes, undo_task_changes	Task journal a rollback.
view_image	Vizuální analýza lokálního obrázku.

Psaní, Vývoj a Počítač

Nástroj	Schopnost
task_plan_status, set_task_plan	Číst nebo vytvořit operační plán.
update_task_step	Aktualizovat stav kroků.
record_task_validation	Zapsat ručně provedenou kontrolu.

Vývoj a Počítač

Nástroj	Schopnost
repo_overview, project_instructions	Mapa repozitáře a hierarchické instrukce.
project_validation_profile, start_project_check	Nabídka a spuštění projektových kontrol.
find_symbol, document_symbols, find_references	Deklarace, symboly dokumentu a rychlá použití.
browser_open, browser_snapshot	Otevřít a sémanticky prohlédnout izolovaný Edge.
browser_fill, browser_click, browser_press, browser_wait	Práce s refs a čekání na změny UI.
browser_select, browser_hover, browser_scroll	Dropdowny, hover a scrolling.
browser_upload, browser_download	Lokální upload a uložení downloadu.
browser_viewport	Desktopový, tabletový nebo mobilní viewport.
browser_console, browser_network	Konzole a síťová diagnostika.
browser_screenshot, browser_close	Vision screenshot a zavření browseru.
git_status, git_diff, git_commit	Lokální Git operace bez automatického push.
run_command	Krátký Bash, PowerShell nebo cmd příkaz.
start_command, poll_command, send_stdin, terminate_command	Persistentní procesy na pozadí.

Počítač

Nástroj	Schopnost
screenshot, get_screen_info	Obrázovka a souřadnice.
move_mouse, click, scroll	Myš a scrolling.
type_text, press_key	Text, klávesy a kombinace.

21. Praktické příklady

Diskuze: Porovnej tyto dva produkční postupy bez coding procedur.

Výzkum: Prozkoumej současný stav virtual production stages. Zachovej rozpory, uveď nejistoty a vyexportuj finální syntézu jako PDF.

Psaní: Přečti treatment v projektu, přepiš druhý akt ve stejném hlasu a ulož DOCX.

Vývoj: Projdi repozitář, reprodukuji chybu, oprav příčinu, spusť testy a shrň změny.

Paměť: Zapamatuj si pro tento projekt, že rendery patří do D:\Show\Delivery.

Připnutí: Připni docs\ARCHITECTURE.md pro tento chat.

Počítač: Udělej screenshot, změň požadované nastavení a ověř výsledek dalším screenshotem.

Export: Ulož existující odpověď jako PDF. Neprováděj nový výzkum.

22. Provozní omezení

Trvalé hranice produktu

Následující funkce jsou **výslovně mimo tento produkt a nikdy nebudou přidány**, pokud vlastník osobně nezmění toto produktové rozhodnutí:

- Language servery nebo distribuční/runtime vrstva LSP. Určeným řešením je vestavěný lehký symbolový index.
- Persistentní interaktivní terminál jako hlavní workflow. Shell zůstává záložní nástroj.
- Paralelní modelová agenta, subagenti nebo multi-model orchestrace. GPU používá jeden lokální model a pracuje sekvenčně.
- Kontext s jedním milionem tokenů. Kontext zůstává v praktických profilech lokální GPU.
- Obecný plugin host, MCP ekosystém nebo široký integrační framework.

Tyto body do Marvin nepatří. Nejsou odloženým roadmap backlogem.

- Jeden velký model zabere téměř celou GPU, proto je inference jednoslotová.
- Čtecí nástroje a procesy na pozadí mohou běžet souběžně.
- Přepnutí modelu/KV restartuje server a dočasnou prompt cache, nikoli historii.
- Kontextová čísla jsou odhady; tokenizace a cena obrázků závisí na modelu.
- Model se může mýlit; ověřovací nástroje nezaručují správnost.
- Computer mode pracuje s primárním monitorem a viditelným UI.
- Web může selhat na login walls, JavaScript-only webech, CAPTCHA nebo blokováných downloadech.
- Veřejné skills před instalací zkontrolujte, protože ovlivňují postup modelu.
- Git commit zůstává lokální bez výslovného požadavku na push.
- Pravidelně zálohujte nenahraditelné projekty a relace.

23. Rozšířené pracovní operace

Čtení celých dokumentů

Modelu můžete zadat konkrétní stránku PDF, blok DOCX, list a oblast Excelu nebo rozsah řádků CSV. Čtecí nástroj vrací také údaj pro navazující část, takže se práce nemusí zastavit na prvních sto řádcích. Excel umí zobrazit uložené hodnoty nebo vzorce; čtecí nástroj sám nepřepočítává vzorce.

Skenované stránky PDF a diagramy lze přes `view_document_page` vykreslit a prohlédnout aktivním modelem s vision. Existující Word soubor upravuje `edit_word_document` se zachováním odstavců, tabulek a stylů textových úseků.

Dohledání původního kontextu

Komprese odpovídá režimu Diskuze, Výzkum, Psaní, Vývoj nebo Počítač. Zpracují se všechny části vstupu a mezisouhrny se ukládají. Původní zprávy lze dohledat přes `search_chat_history` a `read_chat_history`, i když už nejsou v aktivním kontextu.

Výzkum ukládá průběžné poznámky k důkazům. Po přerušení je syntéza může znovu použít. Citace otevře text původního zdroje; nalezené, ale nenačtené položky zůstávají viditelné bez filtru důvěryhodnosti.

Body obnovy

Otevřete **Výsledky > Body obnovy** nebo použijte `/checkpoint` název. Bod zachytí pracovní soubory, nikoli generované závislosti nebo modely a runtime. Úlohový snapshot eviduje také změny provedené příkazy modelu. **Obnovit** oznámí konflikt s pozdější úpravou a nepřepíše ji tiše. Částečné selhání se neoznačí za kompletní obnovení.

Přenos projektu

Použijte **Nastavení > Data a zálohy > Exportovat projekt**. ZIP zahrnuje projektové soubory, projektovou paměť a skilly, historii chatů, přílohy a odkazy na rozhodnutí. **Importovat projekt** vytvoří nový adresář a nové identifikátory chatů a upraví uložené cesty příloh i odkazy na původní konverzace.

Projektový archiv je oddělený od zálohy modelů a runtime. Ta nadále funguje jako pojistka po selhání internetového zdroje; lze ji zvolit i přímo pro offline instalaci.

Vývoj ze zdrojů a vydání

Běžný uživatel volí Minimal nebo Full; Full obsahuje Python 3.12 i závislosti. Node.js není nutný pro provoz aplikace. Vývojář sestaví frontend příkazy `npm --prefix frontend ci` a `npm --prefix frontend run build`. Python obsluhuje výsledný adresář `ui_dist`. Ověřené Windows verze balíčků drží `requirements-windows-py312.lock`.

Původní Gradio rozhraní zůstává pro diagnostiku kompatibility s `MARVIN_LEGACY_UI=1`; standardně se spouští nová pracovní plocha.